



Monitoring & Observability

↓
what?

↓
why?

What is monitoring ?



↪ metrics

- It is the continuous process of observing, measuring, and analyzing system health, performance, and behavior in real-time
- In DevOps, it bridges development and operations by providing feedback loops for faster issue detection and resolution
- Key Goals of monitoring
 - Detect failures early (proactive vs. reactive)
 - Measure user experience (latency, errors, uptime)
 - Support scalability (identify bottlenecks)
 - Enable data-driven decisions (for deployments, rollbacks, auto-scaling)
 - Provide observability across distributed systems (microservices, containers)



Pillars of Observability

■ Metrics

- Numeric data over time (CPU, memory, request rate)
- Examples: Prometheus counters

■ Logs

- Structured/unstructured events → node down, pod restart, service stopped
- Examples: ELK stack (Elasticsearch, Logstash, Kibana)

■ Traces

- Request flow across services
- Examples: Jaeger, Zipkin





Types of Monitoring

- Infrastructure monitoring – CPU, disk, network, server health (e.g., Nagios, Zabbix)
- Application monitoring (APM) – response time, error rates, throughput (e.g., New Relic, Datadog) *SDK*
- Log monitoring – error logs, security logs, audit logs
- Synthetic monitoring – simulate user actions/cron jobs (e.g., Pingdom) → *testing app*
- Real-user monitoring (RUM) – actual user session data
- Security monitoring – intrusion detection, anomaly detection
- Availability/uptime monitoring – health checks, SLA tracking

Common Tools

RoomQL

- Metrics: Prometheus, Graphite, InfluxDB
- Visualization: Grafana, Kibana, Datadog dashboards
- Logging: ELK Stack, Loki, Splunk
- Tracing: Jaeger, Zipkin, AWS X-Ray
- APM: New Relic, Dynatrace, AppDynamics
- Infrastructure: Nagios, Zabbix, Sensu
- Cloud-native: CloudWatch (AWS), Azure Monitor, Stackdriver (GCP)



The Monitoring Workflow

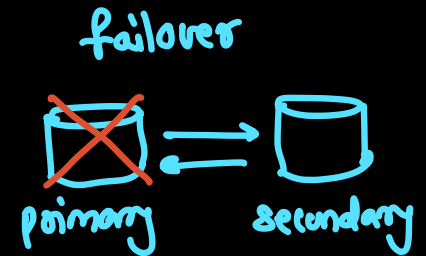


- **Instrument** – add exporters, agents, or SDKs to emit metrics/logs/traces → *decide information to be captured*
- **Collect** – aggregators like Prometheus, Fluentd, or Telegraf
- **Store** – time-series DB (Prometheus), log DB (Elasticsearch)
- **Visualize** – dashboards with Grafana/Kibana
- **Alert** – trigger notifications via Alertmanager, PagerDuty, Slack
- **Incident response** – auto-remediate or page on-call engineers



Best Practices for DevOps Monitoring

- Monitor everything, but prioritize – start with golden signals (Latency, Traffic, Errors, Saturation – Google SRE book)
- Use meaningful thresholds & dynamic alerting – avoid alert fatigue
- Tag/label all data – with environment, service, version, region
- Set up SLIs (Service Level Indicators) & SLOs (Service Level Objectives)
- Automate responses – scaling, failover, restart, logging enhancement
- Keep monitoring config as code – version-controlled dashboards & alerts
- Test your alerts – chaos engineering / game days





Prometheus

capturing metrics

time series db

What is Prometheus



- Prometheus is an open-source systems monitoring and alerting toolkit originally built at SoundCloud in 2012. It was the second project accepted by the Cloud Native Computing Foundation (CNCF) after Kubernetes and one of the first to graduate
- Its core job is to collect metrics (numeric time-series data) from your infrastructure and applications, store them efficiently, and let you query, visualize, and alert on them
- **Pull model**
 - Unlike traditional monitoring systems (like Nagios or Zabbix) where agents actively push data to a central server, Prometheus pulls data
 - The Prometheus server periodically scrapes metrics from HTTP endpoints (typically /metrics) on your applications or services
 - This pull model aligns perfectly with dynamic environments like Kubernetes
 - Prometheus automatically discovers new services and pods without needing to reconfigure a central server every time a container spins up

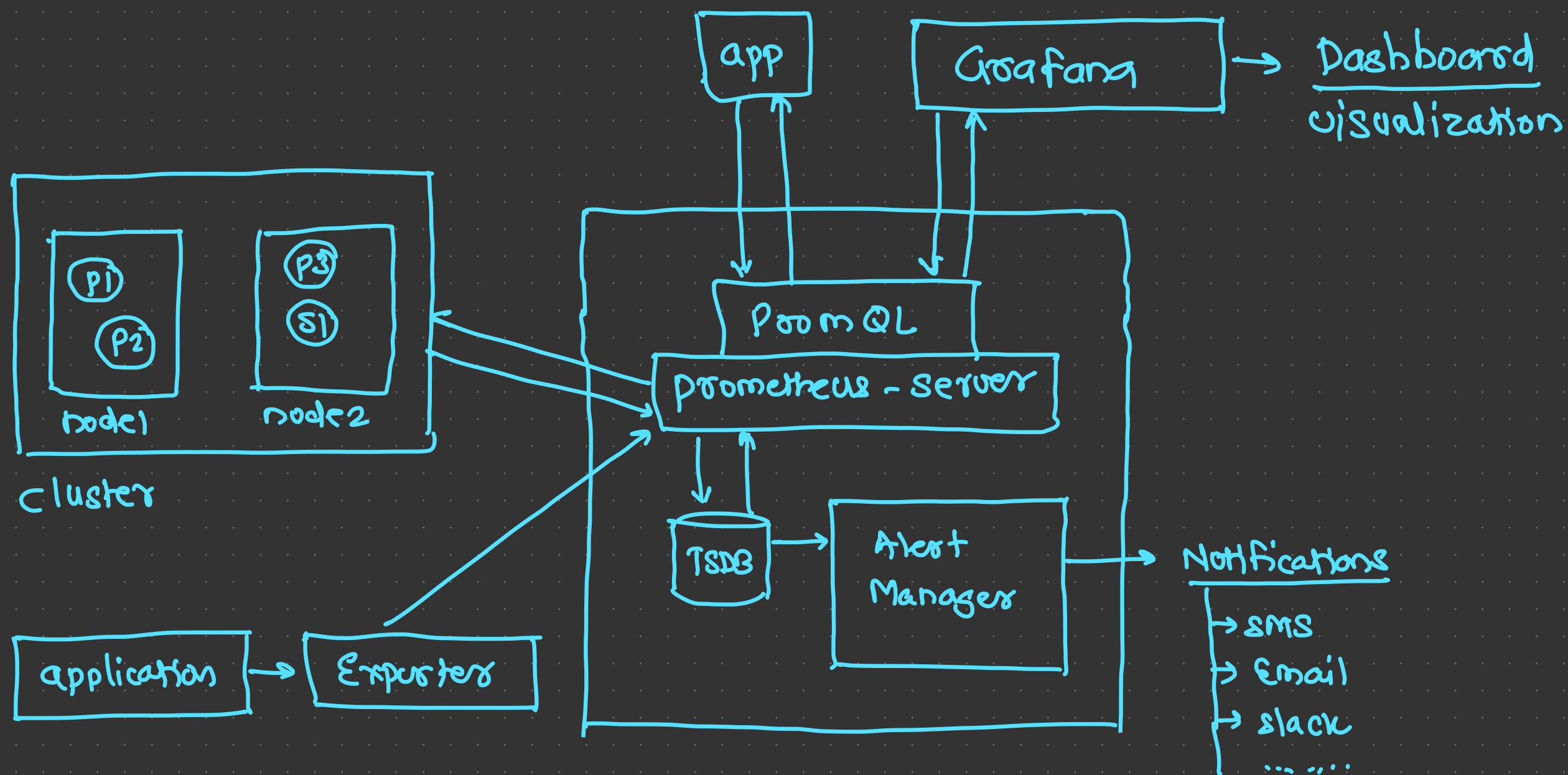
Core Architecture & Components



- **Prometheus Server**
 - time series DB
 - The "brain." It scrapes, stores, and processes metrics
 - It runs the PromQL query language and triggers alerts based on defined rules.
- **Exporters & Client Libraries**
 - The "adapters." Exporters run alongside third-party systems (like a database or Linux server) to convert their internal stats into Prometheus metrics
 - Client Libraries allow you to do the same for your own application code.
- **Alertmanager**
 - The "handler." It takes raw alerts from the Prometheus server, deduplicates, groups, and routes them to notification channels (Slack, PagerDuty, Email)
- **Visualization (Grafana)**
 - Prometheus has a basic UI, but it is almost always paired with Grafana
 - Grafana queries Prometheus using PromQL to build powerful, customizable dashboards

TSDb → Time Series DB

Prometheus Architecture



Prometheus Data Model: Metrics & Labels



- At its heart, Prometheus is a time-series database (TSDB)
- Every piece of data is a timestamped value belonging to a "time series"
- Every time series is uniquely identified by three things:
 - **Metric Name:** Describes what is being measured (e.g., `http_requests_total`, `node_cpu_seconds_total`)
 - **Labels:** Key-value pairs that add dimensions, allowing you to slice and dice your data (e.g., `method="GET"`, `status="200"`, `pod="web-server-1"`)
 - **Timestamp & Value:** The actual data point
- Example: $\curvearrowright$ **metric** **label** **value**
 - `http_requests_total{method="GET", endpoint="/api/users", status="200"} 15423`
- The use of labels is the key to Prometheus's flexibility
- This single metric can be aggregated and filtered across many different dimensions

time	cpu
25/4/26 9:27am	55%
29/4/26 9:28am	60%
⋮	⋮
⋮	⋮
⋮	⋮

Time Series

Key Features



- Multi-data source support (30+ built-in, plus community plugins)
- Rich panel types (time series, bar charts, heatmaps, tables, logs, world map)
- PromQL, LogQL, SQL, and other query editors (including visual query builder)
- Dashboard as code (provisioning via JSON/YAML files)
- Alerting (unified UI for Grafana-managed and data source-managed alerts)
- Annotations (mark deployments, incidents, or events directly on graphs)
- Transformations (pivot, filter, rename, merge data on the fly without changing source)
- User management & RBAC (team-based folder and dashboard permissions)
- Plugins (extend with custom panels, data sources, or apps)

Supported Data Sources



- Time-series databases: Prometheus, Graphite, InfluxDB, Mimir, VictoriaMetrics, TimescaleDB
- Logging systems: Loki, Elasticsearch, Splunk, Datadog
- Tracing backends: Jaeger, Tempo, Zipkin, X-Ray
- Relational databases: PostgreSQL, MySQL, MSSQL
- Cloud monitoring: AWS CloudWatch, Azure Monitor, Google Cloud Monitoring
- Industrial/IoT: Graphite, InfluxDB, MQTT (via plugin)

Metric Types



■ Counter

- A cumulative value that only increases (or resets to zero on restart)
- Total requests, total errors, total bytes sent

■ Gauge

- A value that can go up or down
- Current CPU usage, memory usage, queue length, temperature

■ Histogram

- Samples observations and counts them in configurable buckets. Also provides a sum of all observed values
- Request duration, response size. Excellent for calculating percentiles (p95, p99)

■ Summary

- Similar to Histogram, but calculates quantiles on the client side and pushes them
- Similar use cases as Histogram, but when you need accurate percentiles over a short window

PromQL: The Query Language



- PromQL (Prometheus Query Language) is used to select and aggregate time-series data for dashboards and alerts
- Common PromQL Examples
 - Rate of requests per second over last 5 minutes
 - `rate(http_requests_total[5m])`
 - CPU Usage percentage (excluding idle time)
 - `100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)`
 - 95th percentile response latency
 - `histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))`
 - Prediction (CPU exhaustion in 4 hours)
 - `predict_linear(node_cpu_seconds_total{mode="user"}[1h], 4 * 3600)`



Grafana

What is Grafana



- Grafana is an open-source, interactive data visualization and analytics platform \
- It allows you to query, visualize, alert on, and understand your metrics, logs, and traces no matter where they are stored
- **Core identity:** It is the **universal dashboarding layer** for observability. It does not store data itself but connects to dozens of data sources and presents them in a unified interface

Why Grafana is Critical in DevOps



- **Data scattered across multiple systems**
 - Connects to Prometheus, Loki, CloudWatch, Elasticsearch, etc., into one view
- **Difficult to spot trends in raw numbers**
 - Rich visualizations (graphs, heatmaps, stat panels)
- **Alert fatigue from separate tools**
 - Centralized alerting with unified rules and management
- **No correlation between metrics, logs, and traces**
 - Correlate seamlessly (e.g., click on a spike in metrics → see related logs/traces)
- **Sharing insights across teams**
 - Exportable dashboards, snapshot sharing, public URLs

